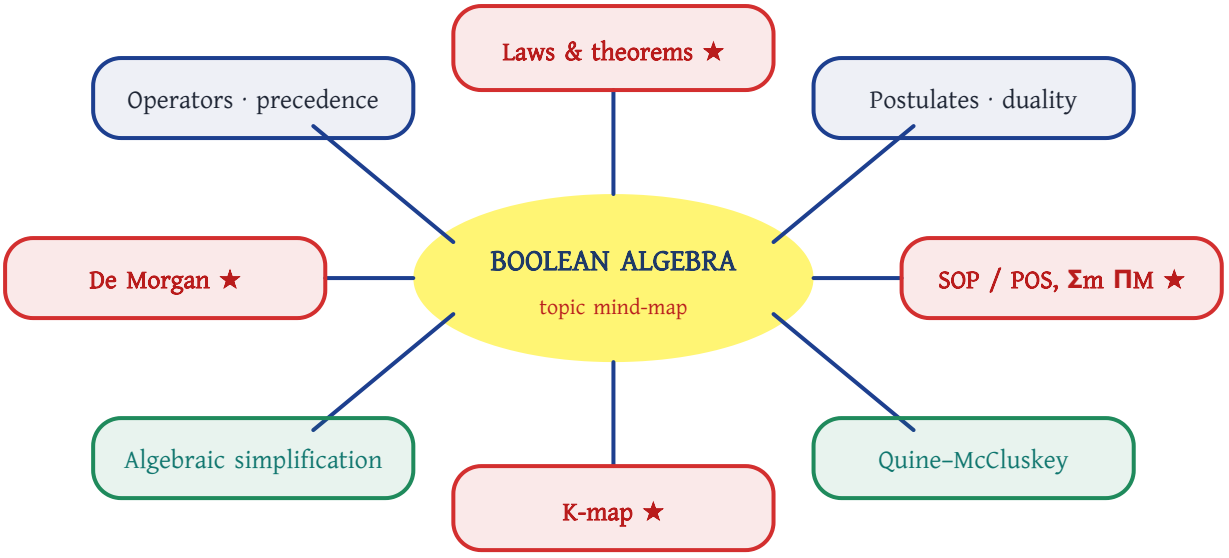
Boolean Algebra

Laws · De Morgan · SOP/POS · K-map · Quine–McCluskey: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

High-yield list

① A + A′B = A + B
② consensus theorem
③ De Morgan & F′
④ Σm ↔ ΠM
⑤ K-map grouping
⑥ don't-cares (X)

Syllabus coverage

"Digital logic: **Boolean algebra**, **De Morgan's Theorems**, logic gates and their truth tables, **canonical forms**, combinational logic circuits, **minimization techniques**". This notebook covers the algebra, canonical forms and minimization (K-map, Quine–McCluskey); gate symbols and gate counts are in the Logic Gates notes.

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Basics, postulates, duality ..... p. 2
✓ Laws & theorems ..... p. 3
✓ De Morgan & complement of F ..... p. 4
✓ Algebraic simplification ..... p. 5
✓ Minterms, maxterms, SOP/POS ... p. 6
✓ K-map rules ..... p. 7
✓ K-map worked examples ..... p. 8
✓ Quine–McCluskey, gates, code ..... p. 9
✓ Night-before sheet ..... p. 10

✓ Practice MCQs + written ..... p. 11

1 What Is Boolean Algebra?

Definition: The algebra of **two values, 0 and 1**, with three basic operations: AND, OR, NOT. Introduced by George Boole (**1854**); Claude Shannon (**1938**) applied it to switching circuits, which made it the mathematics of digital design.

Operation	Symbol	Rule	In C
AND (logical product)	A · B or AB	1 only if **all** inputs are 1	&&
OR (logical sum)	A + B	0 only if **all** inputs are 0	||
NOT (complement)	A′ (also Ā)	flips 0 ↔ 1	!

Precedence: ( ) → NOT → AND → OR

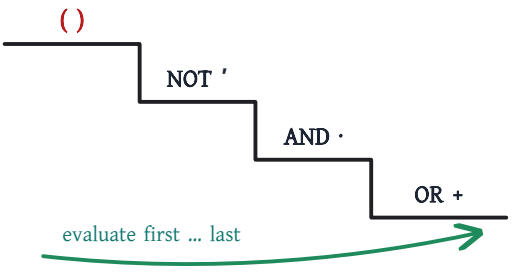
So A + BC′ means A + (B · (C′)), never (A + B)C′.

Counting rules:

n variables → 2^n truth-table rows

n variables → $2^{(2^n)}$ different functions

(n = 2 → 16 functions; n = 3 → 256)



the precedence staircase

Exam trap: Boolean **1 + 1 = 1** (OR). Binary arithmetic **1 + 1 = 10** (sum 0, carry 1). Same symbol, different meaning: read the question carefully.

2 Huntington's Postulates & Duality

1. **Closure** under + and ·
2. **Identity:** A + 0 = A, A · 1 = A
3. **Commutative:** A + B = B + A, AB = BA
4. **Distributive:** A(B + C) = AB + AC **and** A + BC = (A + B)(A + C)
5. **Complement:** A + A′ = 1, A · A′ = 0
6. At least two elements, 0 ≠ 1

Huntington (**1904**): every other law (associative, absorption, De Morgan ...) can be **proved from these six.**

Duality principle: Swap **+ ↔ ·** and **0 ↔ 1** (leave every literal as it is). If an identity is true, its **dual is also true**, so each proof gives you two laws.

Expression	Dual
A + 0 = A	A · 1 = A
A + AB = A	A(A + B) = A
A + BC	A(B + C)

> **Memory trick:** Dual = "**mirror the operators, keep the letters**". Complement = mirror the operators **and** flip every letter.

3 Laws & Theorems (with Duals)

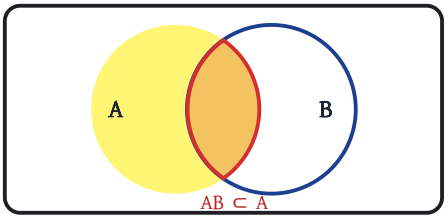
#	Law	Form	Dual form
1	Identity	A + 0 = A	A · 1 = A
2	Null	A + 1 = 1 ★	A · 0 = 0
3	Idempotent	A + A = A ★	A · A = A ★
4	Complement	A + A′ = 1 ★	A · A′ = 0 ★
5	Involution	(A′)′ = A	(self-dual)
6	Commutative	A + B = B + A	AB = BA
7	Associative	A + (B + C) = (A + B) + C	A(BC) = (AB)C
8	Distributive	A(B + C) = AB + AC	**A + BC = (A + B)(A + C) ★**
9	Absorption	A + AB = A ★	A(A + B) = A ★
10	Redundant literal	**A + A′B = A + B ★**	A(A′ + B) = AB ★
11	Consensus	AB + A′C + BC = AB + A′C ★	(A + B)(A′ + C)(B + C) = (A + B)(A′ + C) ★
12	De Morgan	(A + B)′ = A′B′ ★	(AB)′ = A′ + B′ ★

> Laws 9–12 = 80% of simplification marks

★ = no counterpart in school algebra (there, 1 + 1 = 2 and A + A = 2A). Laws 9–12 are the **simplification workhorses**.

> **Proof: absorption ✍️:**
> A + AB = A·1 + AB = A(1 + B) = A · 1 = **A**
> identity, distributive, null, identity

> **Proof: redundant literal ✍️:**
> A + A′B = (A + A′)(A + B) = 1 · (A + B) = **A + B**
> distributive (+ over ·), complement, identity



A + AB: the AB part is already inside A

Memory trick: consensus: In AB + A′C the variable A appears **plain and primed**. Drop it and AND the leftovers → **BC**. That "bridge" term is redundant, so delete it.

Exam trap: A + BC = (A + B)(A + C) is **true** here, though it looks wrong from school algebra. A + A = A, never 2A.

4 De Morgan's Theorems in Depth

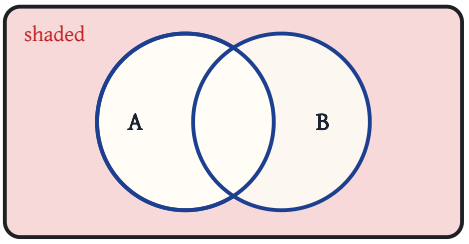
The two theorems: **(A + B)′ = A′ · B′** **(A · B)′ = A′ + B′**

n variables: (A + B + C + ...)′ = A′B′C′... · (ABC...)′ = A′ + B′ + C′ + ...

Proof by truth table

A	B	(A+B)′	A′B′	(AB)′	A′+B′
0	0	1	1	1	1
0	1	0	0	1	1
1	0	0	0	1	1
1	1	0	0	0	0

Columns 3 = 4 and 5 = 6 in every row → **proved**.



(A + B)′ = A′ · B′: outside both

a bar over the whole ≠ a bar over each letter!

Complement of a whole function, F′

1. Put brackets round every AND term
2. Swap + ↔ · (and 0 ↔ 1): this is the dual
3. Complement **every literal** (A ↔ A′)
4. Simplify if asked

Memory trick: F′ = **dual of F with every literal flipped**. "Break the long bar, change the sign under the break."

Worked example 1 ✍️: F = AB′ + A′C → F′ = **(A′ + B)(A + C′)**

Expand: A′A + A′C′ + AB + BC′ = A′C′ + AB + BC′ = **A′C′ + AB** (BC′ is the consensus of A′C′ and AB)

Worked example 2 ✍️: F = x(y′z + yz′) → F′ = x′ + (y + z′)(y′ + z) = **x′ + yz + y′z′** (= x′ + (y ⊙ z))

Worked example 3 ✍️: (A + BC)′ = A′ · (BC)′ = A′(B′ + C′) = **A′B′ + A′C′** De Morgan twice, then distributive

Exam trap: $(A + B)' \neq A' + B'$ and $(AB)' \neq A'B'$. Flipping the letters **without** swapping the operator is the most common mistake.

5 Algebraic Simplification

Goal: Fewest **terms**, then fewest **literals** (a literal = a variable or its complement in a term). Fewer literals → **fewer gates, less delay, lower cost**. Name the law at every step: examiners mark the steps.

Example 1 ✍️: $F = AB + AB' + A'B$

1. $AB + AB' = A(B + B') = A \cdot 1 = A$ distributive, complement, identity
2. $F = A + A'B =$ **$A + B$** redundant literal (6 literals → 2)

Example 2 ✍️: $F = (A + B)(A + B') = A + BB' = A + 0 =$ **A** distributive (+ over ·), complement, identity

Example 3 ✍️: $F = A'B'C + A'BC + AB'C$

1. $A'B'C + A'BC = A'C(B' + B) = A'C$ distributive, complement
2. $F = A'C + AB'C = C(A' + AB') = C(A' + B')$ redundant literal
3. $F =$ **$A'C + B'C$** (9 literals → 4; factored $C(A' + B') = 3$)

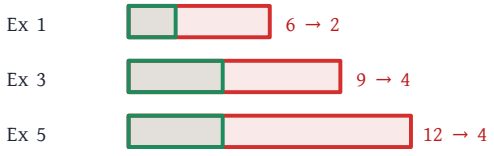
Example 4: consensus ✍️: $F = XY + X'Z + YZ$

1. $YZ = YZ(X + X') = XYZ + X'YZ$ identity, complement, distributive
2. $XY + XYZ = XY$ and $X'Z + X'YZ = X'Z$ absorption
3. $F =$ **$XY + X'Z$**

Example 5 ✍️: $F = A'BC + AB'C' + ABC' + ABC$

1. $A'BC + ABC = BC(A' + A) = BC$
2. $AB'C' + ABC' = AC'(B' + B) = AC'$
3. $F =$ **$BC + AC'$** (12 literals → 4)

Look for, in this order: ① $X + X'$ pairs (combine) ② a term inside another (absorption) ③ $X + X'Y$ (drop X') ④ a consensus "bridge" term (delete).



literals before (red) vs after (green)

6 Minterms, Maxterms & Canonical Forms

A B C	Minterm	m_i	Maxterm	M_i
0 0 0	A′B′C′	m0	A + B + C	M0
0 0 1	A′B′C	m1	A + B + C′	M1
0 1 0	A′BC′	m2	A + B′ + C	M2
0 1 1	A′BC	m3	A + B′ + C′	M3
1 0 0	AB′C′	m4	A′ + B + C	M4
1 0 1	AB′C	m5	A′ + B + C′	M5
1 1 0	ABC′	m6	A′ + B′ + C	M6
1 1 1	ABC	m7	A′ + B′ + C′	M7

Minterm m_i: AND of **all** variables; = 1 for exactly one row. **Canonical SOP** = OR of minterms = **Σm**

Maxterm M_i: OR of **all** variables; = 0 for exactly one row. **Canonical POS** = AND of maxterms = **ΠM**

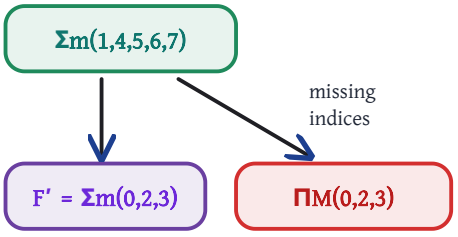
Key relation: $m_i' = M_i$ · F = Σm(...) = ΠM(the **missing** indices) · F′ = Σm(missing) · **#m + #M = 2^n**

> index = the row number in binary: ABC = 101 → m5

Standard SOP/POS (terms may miss variables, e.g. A + B′C) vs **canonical** (every term has every variable).

Worked example ✍️: Expand F = A + B′C to canonical form.

1. A = A(B + B′)(C + C′) = m4 + m5 + m6 + m7
2. B′C = (A + A′)B′C = m5 + m1
3. F = **Σm(1, 4, 5, 6, 7)** = ΠM(0, 2, 3)
4. F′ = Σm(0, 2, 3)



Memory trick: Minterm: 1 → plain, 0 → primed. Maxterm: **exactly the opposite** (0 → plain).

Exam trap: 3-variable F = Σm(1, 3, 5) has **8 − 3 = 5** maxterms in its ΠM form.

7 Karnaugh Map (K-map): the Rules

Definition: A K-map (Karnaugh, **1953**) is the truth table redrawn as a grid in **Gray-code order**, so neighbouring cells differ in **exactly one variable**. Grouping neighbours removes that variable.

2 variables

A \ B	0	1
0	0	1
1	2	3

3 variables

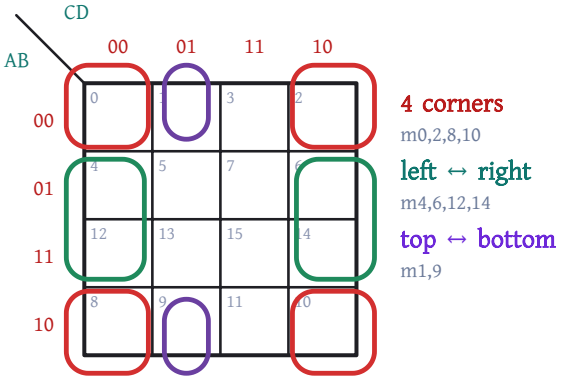
A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

4 variables

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

cell numbers = minterm indices · columns run **00, 01, 11, 10** (not 10, 11)

1. Group the **1s** for SOP (the **0s** for POS); an X (don't-care) may join any group
2. Group sizes: **1, 2, 4, 8, 16**; rectangles only, no L-shapes or diagonals
3. Make groups **as large as possible**, and use **as few** as possible
4. Groups may **overlap**; every 1 must be covered
5. Edges wrap: left ↔ right, top ↔ bottom; the 4 corners are neighbours
6. Read a group: keep the variables that stay constant



Group (4-var)	Literals left
1 cell	4
2 cells	3
4 cells	2
8 cells	1
16 cells	F = 1

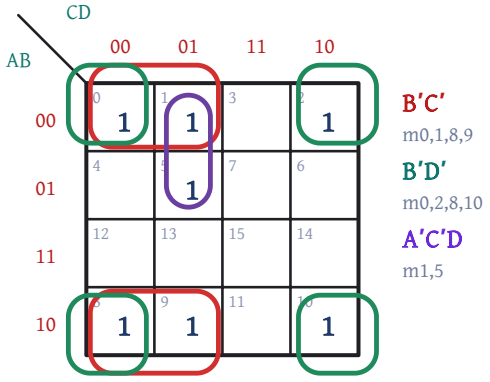
Prime implicant (PI): a group that cannot be made larger

Essential PI: a PI covering a 1 that **no other** PI covers → it **must** appear in the answer

Exam trap: "Adjacent" means **one bit** different: m3 (0011) and m2 (0010) are neighbours; m1 (0001) and m2 (0010) are **not**, though 1 and 2 are consecutive numbers.

8 K-map Worked Examples

(a) F(A, B, C, D) = Σm(0, 1, 2, 5, 8, 9, 10)

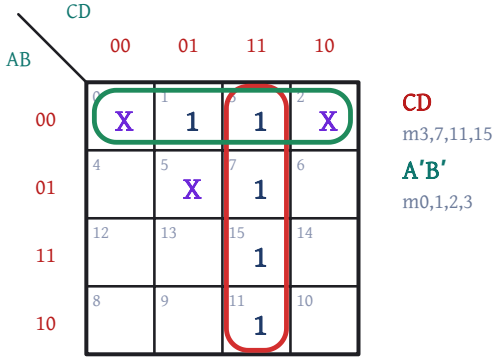


- **B′C′**: quad m0, 1, 8, 9 (top and bottom rows wrap)
- **B′D′**: the four corners m0, 2, 8, 10
- **A′C′D**: pair m1, 5

F = **B′C′ + B′D′ + A′C′D**

All three are **essential**: m9, m10 and m5 are each covered by one group only. 7 literals.

(b) With don't-cares: F = Σm(1, 3, 7, 11, 15) + d(0, 2, 5)

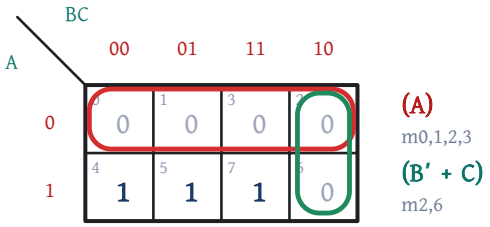


- **CD**: column CD = 11 (m3, 7, 11, 15)
- **A′B′**: row AB = 00, using X at m0 and m2

F = **CD + A′B′**

CD + A′D (using X at m5) is **equally minimal**: both have 4 literals. X5 is simply left out of the first answer.

(c) POS by grouping 0s: F(A, B, C) = ΠM(0, 1, 2, 3, 6)



- Row A = 0 (four 0s) → sum term **(A)**
- m2, m6 (B = 1, C = 0) → **(B′ + C)**
- In a 0-group: a variable that is 0 stays plain, 1 is primed

F = **A(B′ + C)**

Memory trick: Inside a group, a variable that **changes** disappears; a variable that stays constant is kept.

Exam trap: Use an X only if it makes a group **bigger**; don't-cares never have to be covered.

9 Quine–McCluskey (Tabular) Method

Idea: A systematic, **computer-friendly** version of the K-map (Quine 1952, McCluskey 1956). It works for any number of variables.

1. Write the minterms in binary and **group them by the number of 1s**
2. Combine terms in neighbouring groups that differ in **one bit**; write – for that bit and tick both
3. Repeat until nothing combines; **unticked terms = prime implicants**
4. PI chart: take the essential PIs, then cover the rest with the fewest PIs (Petrick's method)

Mini example ✍️: F = Σm(0, 1, 2, 3, 7)

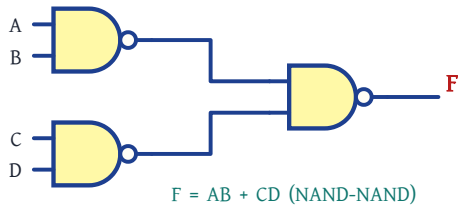
Group	Column I	Column II	Column III
0 ones	0: 000 ✓	0,1: 00– ✓ · 0,2: 0–0 ✓	**0,1,2,3: 0––**
1 one	1: 001 ✓ · 2: 010 ✓	1,3: 0–1 ✓ · 2,3: 01– ✓	
2 ones	3: 011 ✓	**3,7: –11**	
3 ones	7: 111 ✓		

Unticked = PIs: 0–– = A′ and –11 = BC; both essential (m0 and m7) → F = **A′ + BC**

	K-map	Quine–McCluskey
Practical size	up to 4 (5–6 with care) variables	any number
Method	visual, by hand	tabular, mechanical
Best for	exams, small circuits	software / CAD tools

SOP → AND-OR → **NAND-NAND** · **POS** → OR-AND → **NOR-NOR** (same two levels; see Logic Gates notes, sheet 7).

Cost ≈ number of gates + gate inputs: that is why we minimize literals first.



IT link: The same laws simplify code: `!(age < 18 || age > 60) = age >= 18 && age <= 60` (De Morgan) · `x || (x && y) = x` (absorption) · SQL `dept = 'IT' OR (dept = 'IT' AND pay > 50000) = dept = 'IT'`

Night-Before-the-Exam Sheet

Fact sheet:

Precedence: ( ) → NOT → AND → OR

n vars → 2^n rows · $2^{(2^n)}$ functions

A + 1 = 1 · A · 0 = 0 · A + A = A

A + A′ = 1 · A · A′ = 0 · A″ = A

A + AB = A · A(A + B) = A

A + A′B = A + B · A(A′ + B) = AB

A + BC = (A + B)(A + C)

AB + A′C + BC = AB + A′C

(A + B)′ = A′B′ · (AB)′ = A′ + B′

Dual: + ↔ ·, 0 ↔ 1 (letters stay)

$m_i' = M_i \cdot \Sigma m(...) = \Pi M(\text{missing})$

K-map order 00, 01, 11, 10

Group of 2^k cells removes k variables

SOP → NAND-NAND · POS → NOR-NOR

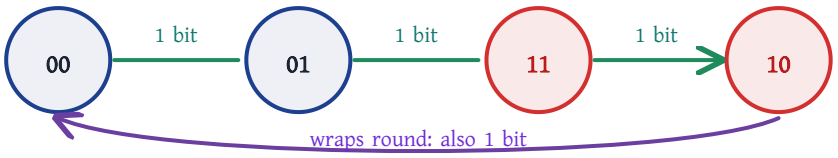
Top 12 one-liners for MCQs

1. Boolean algebra: Boole, 1854
2. Applied to switching: Shannon, 1938
3. A + AB = A (absorption)
4. A + A′B = A + B
5. Dual of A + BC is A(B + C)
6. 4-variable K-map has 16 cells
7. K-map groups: 1, 2, 4, 8, 16 only

8. Four corners of a 4-var map = one group
9. $\Sigma m(0, 2, 4, 6)$ of A, B, C = C'
10. Essential PI must be in every answer
11. QM = tabular method, for many variables
12. #minterms + #maxterms = 2^n

Top traps: Boolean 1 + 1 = 1, binary 1 + 1 = 10 · dual ≠ complement · $(A + B)' \neq A' + B'$ · adjacency is by one bit, not by consecutive numbers · don't-cares need not be covered · minimal answers may not be unique

All tricks together: Precedence staircase · dual = mirror operators, keep letters · consensus "bridge" term is redundant · break the bar, change the sign · minterm 0 → primed, maxterm 0 → plain · in a group, the changing variable disappears



Gray-code order: every step, including the wrap, changes one bit

✓ Practice MCQs (exam pattern, 10 marks)

1. A + 1 equals:
   (a) A
   (b) 0
   (c) A'
   (d) 1
2. A + AB simplifies to:
   (a) B
   (b) AB
   (c) A
   (d) A + B
3. $(A + B)'$ equals:
   (a) $A'B'$
   (b) $A' + B'$
   (c) AB
   (d) $A'B$
4. $F(A, B, C) = \Sigma m(0, 2, 4, 6)$ simplifies to:
   (a) A'
   (b) $B'C'$
   (c) $A'C'$
   (d) C'
5. $AB + A'C + BC$ simplifies to:
   (a) AB + C
   (b) $AB + A'C$
   (c) A + C
   (d) B + C

6. A + A′B equals:

(a) A + B

(b) AB

(c) A′ + B

(d) B

7. The dual of A + BC is:

(a) A′(B′ + C′)

(b) (A + B)(A + C)

(c) A(B + C)

(d) A + B + C

8. Number of cells in a 4-variable K-map:

(a) 8

(b) 16

(c) 4

(d) 32

9. Which group size is NOT allowed in a K-map?

(a) 2

(b) 4

(c) 8

(d) 6

10. For 3 variables, F = Σm(1, 3, 5) has how many maxterms in ΠM form?

(a) 3

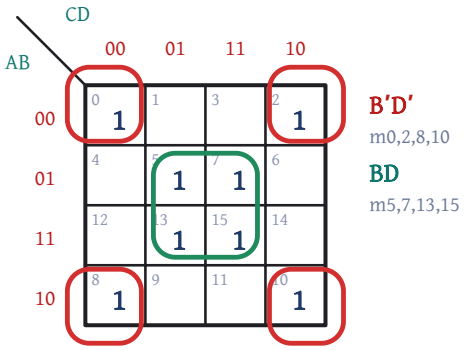
(b) 5

(c) 8

(d) 2

Answer key → 1-d · 2-c · 3-a · 4-d · 5-b · 6-a · 7-c · 8-b · 9-d · 10-b

Written question: Minimize F(A, B, C, D) = Σm(0, 2, 5, 7, 8, 10, 13, 15) with a K-map and implement it with NAND gates only. 8 marks



1. Corners m0, 2, 8, 10 → **B′D′**; centre m5, 7, 13, 15 → **BD** 3
2. F = **B′D′ + BD** = B ⊙ D (XNOR); A and C drop out 2
3. NAND-NAND: F = ((B′D′)′ · (BD)′)′, with B′ = NAND(B, B), D′ = NAND(D, D) 2
4. Total: 2 inverters + 2 + 1 = **5 NAND gates** 1

Before the exam ✓

✓ Can state all 12 laws with their duals

✓ Can find F′ and the dual without mixing them

✓ Can convert Σm ↔ ΠM and expand to canonical

✓ Can group 3- and 4-variable K-maps with X

> **Also practise (written):**
> ① Prove the consensus theorem
> ② Σm(1,3,7,11,15) + d(0,2,5)
> ③ Minimize a POS with 0-groups
> ④ QM for Σm(0,1,2,5,6,7)